

Mobile and Embedded Computing

Lecture 5: server vs client-side execution, serverless vs VPS, Go backends & Firebase

Who's teaching this course

Dinu-Ștefan Rusu

dinu_stefan.rusu@upb.ro

Microsoft Teams course channel

rusudinu.com

45+

Flutter apps shipped

400k

installs across them

Appeared in, spoken at & partnered with



TODAY

What you should leave with



Split client and server

What has to run on a machine you control, and why the client is untrusted input



Serverless or a VPS

Cold starts, scaling and cost shape, and the case where each one wins



Talk HTTP properly

Safe vs idempotent, timeouts and retries with jitter, in real dio code



Stop hand-writing JSON

@JsonSerializable and build_runner, then Firebase from a Flutter app

SERVER & CLIENT

Where does the code run?

The same feature costs different things on either side of the wire

Server-side and client-side execution

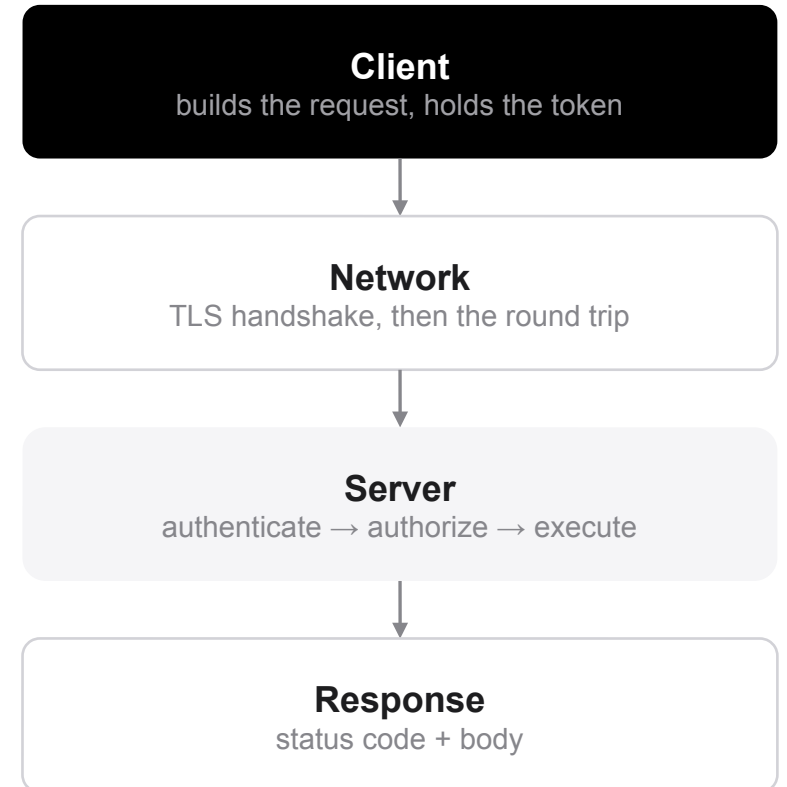
Client-side code runs on the user's device: rendering, local state, caching, optimistic updates, and the checks that make a form feel fast.

Server-side code runs on a machine you control: secrets, durable storage, authorization, billing, heavy compute, and anything that must be identical for every user.

Between them sits one request/response cycle, and on mobile it is expensive: 40–200 ms round trip on a good connection, seconds on a bad one, nothing at all in a tunnel.

So the split is a latency decision as much as a security one.

Put work on the client for latency. Put it on the server when the answer has to be correct for every user.



The client is untrusted input

Everything that arrives from a client is user input, including the fields your own app filled in.

An APK can be unpacked, release Dart can be inspected, and any user can proxy their own device's traffic. There is no secret you can ship inside a client.

Client-side validation is a UX feature: an instant “that is not an email”, a disabled button, no wasted round trip.

The server repeats every check it cares about, because it is the only copy nobody can edit.

Server-side only

API keys, and anything signed with one

Who may read or write which record

Prices, totals, discounts, balances

Rate limits and quotas

Anything you would be unhappy to see edited in a debugger

Validate on the client for speed, on the server for correctness. Any rule enforced only in the app can be bypassed.

WHERE THE SERVER RUNS

Serverless vs a VPS

Two cost shapes, two failure modes and two different sets of operational work

Functions as a service

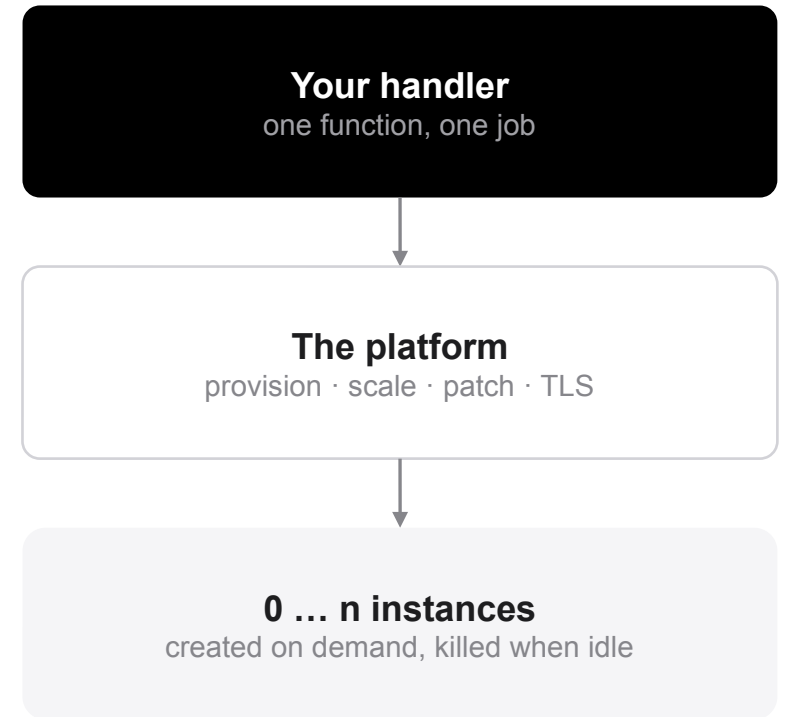
You deploy a function, not a machine. The platform provisions, scales, patches and terminates TLS for you.

It creates instances on demand and destroys them when traffic stops, scaling all the way down to zero.

Billing is per invocation plus resource-time (GB-seconds), so no traffic means no bill.

AWS Lambda, Google Cloud Run functions (the former Cloud Functions), Azure Functions, Cloudflare Workers, and Firebase Cloud Functions, which is the same model packaged for Firebase.

You stop operating servers. In exchange you work within the platform's limits.



Cold starts: the latency of the first request

0 ms

Warm instance

the container is already up, which covers most requests

10–50 ms

Go or Rust binary

nothing to boot: one static executable and a syscall

0.3–2 s

JVM, .NET, large Node graphs

runtime start, plus every dependency your handler imports

A cold start is the platform building a new instance because no warm one was free: pull the image, start the runtime, run your top-level code, then finally your handler.

It lands on the first user after a period of no traffic, and again on every traffic spike.

Mitigations all cost something.

Minimum instances, or provisioned concurrency, means paying for idle capacity, which is the cost serverless removes in the first place. Smaller packages and a fast-starting runtime cost nothing extra, which is one reason for the Go section later in this lecture.

Renting the whole machine

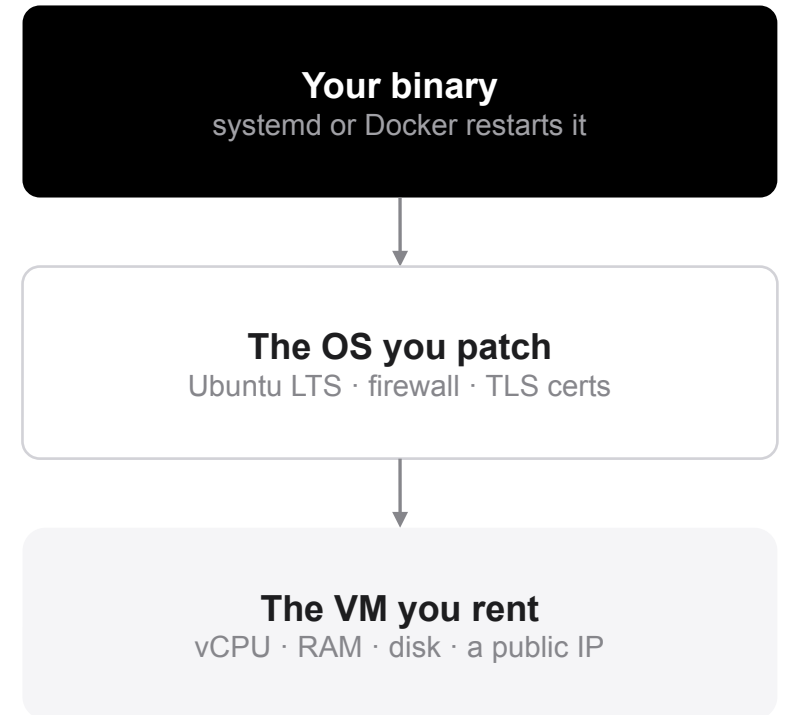
You rent a virtual machine from Hetzner Cloud, DigitalOcean or OVH, and what you get is root, an IP address, and nothing else.

Everything above the hypervisor is yours: OS updates, firewall rules, TLS certificates, process supervision, backups, monitoring.

Hetzner adds firewalls, snapshots and floating IPs. A floating IP is a movable address you can repoint at a fresh box during an upgrade.

Billing is hourly or monthly for the machine, not for the requests it serves. An idle VPS costs what a busy one costs, which helps when traffic is steady and hurts when it is not.

Nobody patches it for you. In exchange, no platform limits what you can run.



The same backend, two cost shapes

	Serverless (FaaS)	VPS
Scaling	automatic, zero to peak and back	you resize the box by hand, in advance
Cost when idle	nothing	the full monthly price
Cost under load	per invocation + GB-second	flat, until you outgrow the machine
First request	a cold start, unless you pay to avoid it	already warm
You operate	your handler	OS, runtime, TLS, backups, monitoring
Long-running work	capped by the platform timeout	runs as long as you let it
Wins when	traffic is spiky, small or unknown	traffic is steady, or state lives on disk

Pick by cost shape and operational load. Spiky, small or brand new → serverless. Steady and predictable → one small VPS is cheaper and simpler. → Lecture 12, gRPC on the same backend

Writing code that survives a FaaS runtime



Ship a small package

Every megabyte and every import is cold-start time. Trim the dependency list before you tune anything else.



Initialize outside the handler

Database pools and HTTP clients go at module scope, so warm invocations reuse them instead of rebuilding them.



Keep the handler stateless

Local disk and memory vanish between invocations. State belongs in a database, a cache or a bucket.



Make it idempotent

The platform retries on its own. A duplicate delivery must not charge the card a second time.



Respect the timeout

Work that can exceed it belongs on a queue with a job record, not in a function with a longer limit.



Know your concurrency

Lambda serves one request per instance; Cloud Run serves many at once, which changes what is safe to share.

Running a box you will not regret



Rebuild it from a script

A Dockerfile plus one provisioning script. The machine should be reproducible, never hand-tuned and remembered.



Run it as a service

systemd or Docker with a restart policy. A crash at 03:00 has to recover without manual intervention.



Lock down access

Keys-only SSH, no root login, firewall down to 80/443, unattended security upgrades switched on.



Terminate TLS properly

Caddy or nginx in front, with automatic certificates. Never serve an API over plain HTTP, not even internally.



Test the restore

Snapshots plus an off-box copy of the database. A backup you have never restored is unverified.



Make it page you

Uptime check, log shipping, disk-full alert. Nothing on a bare VPS tells you it went down by default.

GO BACKENDS

A backend that fits in one binary

One static binary, milliseconds to start, deployable anywhere

Why Go for a small mobile backend

One static binary.

go build produces a single executable with no runtime, no interpreter and no dependency folder. Copy it to the server, or put it in a 15 MB container.

It starts in milliseconds.

Nothing to boot means cold starts on a scale-to-zero platform stay short.

It cross-compiles from your laptop.

One environment variable retargets the build: a Linux/arm64 server, or a Raspberry Pi on the same desk.

The standard library is already the server.

net/http, encoding/json and database/sql, plus the goroutines you already met in Lecture 3, one per request. → TinyGo on a microcontroller, Lecture 12

```
# one static binary, nothing else on the box
CGO_ENABLED=0 GOOS=linux GOARCH=arm64 \
    go build -o server ./cmd/server
```

```
# VPS: copy it, let systemd own it
scp server root@vps:/usr/local/bin/
ssh root@vps 'systemctl restart api'
```

```
# serverless: the same binary, in a tiny image
gcloud run deploy api --source .
```

A small binary that starts fast. Those are the properties that matter for the service a mobile app talks to.

A handler, and the Dart call that hits it

Go: standard library only

```
type Reading struct {
    DeviceID string `json:"device_id"`
    TempC     float64 `json:"temp_c"`
}

func handleReading(w http.ResponseWriter, r *http.Request) {
    var in Reading
    if err := json.NewDecoder(r.Body).Decode(&in); err != nil {
        http.Error(w, "bad json", http.StatusBadRequest)
        return
    }
    w.WriteHeader(http.StatusCreated)
    json.NewEncoder(w).Encode(in) // echo the stored value
}

func main() {
    http.HandleFunc("POST /readings", handleReading)
    log.Fatal(http.ListenAndServe(":8080", nil))
}
```

Dart: the client side of the same call

```
final dio = Dio(BaseOptions(
    baseUrl: 'https://api.example.com',
));

final res = await dio.post(
    '/readings',
    data: {'device_id': id, 'temp_c': 21.5},
);

final saved = Reading.fromJson(res.data);
```

The JSON tags on the Go struct and the `@JsonKey` names on the Dart model are the contract. Change one, break the other.

Routing patterns like "POST /readings" are built into net/http since Go 1.22, so no framework is required.

201 Created, not 200. The status code is part of the API.

HTTP FROM FLUTTER

Requests, failures and JSON

Methods, timeouts, retries, and generated JSON instead of a hand-written fromJson

Safe is not the same as idempotent

Safe: the request does not modify state. **Idempotent:** sending it twice has the same effect as sending it once.

	Safe	Idempotent	What you use it for
GET	yes	yes	read a resource
HEAD	yes	yes	the headers only: size, caching, existence
PUT	no	yes	replace the resource at a URL you chose
DELETE	no	yes	remove it; the second call still leaves it gone
POST	no	no	create, or anything that is none of the above
PATCH	no	no	partial update, not idempotent in general

Every safe method is idempotent; the reverse is false. PUT and DELETE do modify state, but a repeat does not change it further. That is what makes them safe to retry, and POST not.

dio: a request, and its failure paths

```
final dio = Dio(BaseOptions(  
  baseUrl: 'https://api.example.com',  
  connectTimeout: const Duration(seconds: 5),  
  receiveTimeout: const Duration(seconds: 10),  
));  
  
Future<Reading> fetchReading(String id) async {  
  try {  
    final res = await dio.get('/readings/$id');  
    return Reading.fromJson(res.data as Map<String, dynamic>);  
  } on DioException catch (e) {  
    if (e.type == DioExceptionType.connectionTimeout) {  
      throw ApiFailure('server did not answer in time');  
    }  
    if (e.response?.statusCode == 404) throw NotFound(id);  
    throw ApiFailure(e.message ?? 'network error');  
  }  
}
```

http or dio?

package:http is the minimal client: one function per request, and you assemble everything else yourself.

package:dio adds base URLs, timeouts, interceptors (auth headers, logging, refresh), cancellation and multipart uploads.

Set timeouts explicitly. The default on a mobile network is effectively no timeout at all, so a request can hang until the user closes the app.

Map DioException to your own error type at the edge. The UI layer should never see a package's exception class.

Retry with exponential backoff and jitter

```
Future<T> withRetry<T>(Future<T> Function() call) async {  
    const base = 200;                // milliseconds  
    for (var attempt = 0; ; attempt++) {  
        try {  
            return await call();  
        } on DioException catch (e) {  
            if (!_retryable(e) || attempt == 4) rethrow;  
            final expo = base * (1 << attempt); // 200,400,800...  
            final wait = Random().nextInt(expo); // full jitter  
            await Future.delayed(Duration(milliseconds: wait));  
        }  
    }  
}
```

	Backoff	Actual wait
Try 1	200 ms	0 – 200 ms
Try 2	400 ms	0 – 400 ms
Try 3	800 ms	0 – 800 ms
Try 4	1600 ms	0 – 1600 ms
Try 5	3200 ms	0 – 3200 ms

Retry: 408, 429, 500, 502, 503, 504, timeouts, connection resets.

Never: 400, 401, 403, 404, 422: the answer will not change. And never repeat a POST without an idempotency key the server remembers.

Jitter matters as much as the doubling. Doubling alone still lines every client up on the same millisecond.

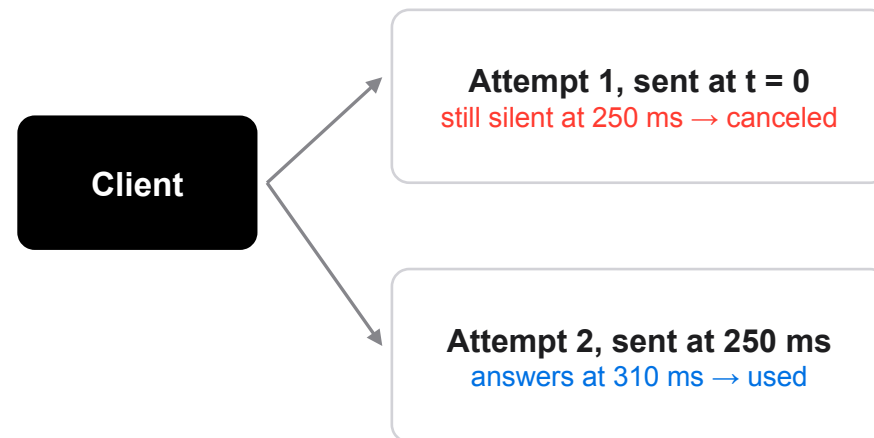
Thundering herds and hedged requests

When a service fails briefly, ten thousand clients fail at once and every one of them retries after exactly one second. The retry wave is larger than the original load. That is a thundering herd, and it turns a brief outage into a long one.

Full jitter spreads those retries across the whole interval, so the recovering server sees load ramp up instead of arriving all at once.

Hedging is the opposite problem.

A small fraction of requests are simply slow (p95, p99). Send a second copy after a short delay, take whichever answers first, cancel the other. gRPC has hedgingPolicy built in.



Cost: one extra request on the slow tail. Cap it: one hedge, only past p95, and only for safe or idempotent calls.

Retries fix failures. Hedges fix slowness. Both multiply load, so both need a cap.

@JsonSerializable and the generated half

```
// lib/models/reading.dart
part 'reading.g.dart';          // the generated half

@JsonSerializable()
class Reading {
  const Reading({required this.deviceId, required this.tempC});

  @JsonKey(name: 'device_id') final String deviceId;
  @JsonKey(name: 'temp_c')     final double tempC;

  factory Reading.fromJson(Map<String, dynamic> json) =>
    _$ReadingFromJson(json);

  Map<String, dynamic> toJson() => _$ReadingToJson(this);
}
```

```
# writes lib/models/reading.g.dart
dart run build_runner build \
  --delete-conflicting-outputs

# regenerate on every save
dart run build_runner watch
```

The round trip: `Reading.fromJson(jsonDecode(body))` in, `jsonEncode(reading.toJson())` out.

`part` links your file to its generated twin; the `_$` functions live there. Never edit a `.g.dart`.

Hand-written mapping is where a renamed field silently becomes null. The generator fails the build instead.

Commit the `.g.dart` files or generate them in CI. Pick one and write it down.

Two files, one model. You write the class; `build_runner` writes the mapping code.

The generators you will actually add

	What it generates	Why you would add it
json_serializable	fromJson / toJson for annotated classes	every project with an API has this one
freezed	immutable classes, copyWith, sealed unions	modeling loading / data / error states → Lecture 4
retrofit	a typed dio client from an abstract class	one place where the endpoints are declared
riverpod_generator	providers from annotated functions	less boilerplate, and typos become build errors
drift	type-checked SQL, tables and migrations	local storage that survives offline → Lecture 10
mockito	mocks for your own interfaces	tests that do not touch the network

build_runner is an ordinary build step. It is slow on large projects, it fails when generated files go stale (--delete-conflicting-outputs), and it belongs in CI. It is still worth adding.

FIREBASE

A backend you configure instead of writing

flutterfire, Firestore, and the rules that are the only server you get

Wiring Firebase into a Flutter app

```
# once per machine
dart pub global activate flutterfire_cli

# registers iOS/Android/web apps and writes
# lib/firebase_options.dart for you
flutterfire configure --project=my-app

flutter pub add firebase_core cloud_firestore firebase_auth
```

```
// main.dart: before runApp, always
WidgetsFlutterBinding.ensureInitialized();
await Firebase.initializeApp(
  options: DefaultFirebaseOptions.currentPlatform,
);
runApp(const MyApp());
```

The packages, by name

firebase_core: required by every other one; it is what `initializeApp` comes from

cloud_firestore: the document database and its snapshot streams

firebase_auth: sign-in, tokens, and the uid your rules compare against → Lecture 7

firebase_analytics · **firebase_crashlytics**: Dart APIs, not Android Bundle code

firebase_ai: Gemini from Flutter; it replaces the deprecated **google_generative_ai** → Lecture 9

firebase_messaging: FCM push, plus a background handler you must register at top level

Firestore: read, write, and the rule that guards it

```
final db = FirebaseFirestore.instance;

// write: merge, so other fields survive
await db.collection('readings').doc(uid).set(
    {'temp_c': 21.5, 'at': FieldValue.serverTimestamp()},
    SetOptions(merge: true),
);

// read once
final snap = await db.collection('readings').doc(uid).get();
final data = snap.data();

// live: a Stream, so StreamBuilder can own it
db.collection('readings').doc(uid).snapshots().listen(...);
```

```
match /readings/{userId} {
  // NOT: if request.auth != null
  // that is every signed-in user,
  // writing to everybody's document
  allow read, write: if
    request.auth != null &&
    request.auth.uid == userId;
}
```

Firestore rules run on Google's side, so they are the only server-side check you have here.

Reads are billed per document: a query that returns 500 docs costs 500 reads, on every rebuild.

Never store a password or an API key in a document. → Lecture 7

The rules file is your backend. Everything from slide five applies to it.

Cloud Functions, and where Firebase stops

Cloud Functions for Firebase are the FaaS from section two, wired to Firebase triggers: a Firestore write, a new Auth user, a scheduled job, or an HTTPS callable your app invokes directly.

They are written in TypeScript, JavaScript or Python, not Dart. That is a second language in your repository, and a real cost.

Use them for exactly what the client must not do: verifying a purchase receipt, sending FCM to someone else's device, calling a paid API with your secret key.

Where Firebase stops:

joins, aggregate reporting, anything you want to query in three ways next month, and any bill you need to predict. That is where the Go backend from section three fits.

```
// calling one from Flutter
final fn = FirebaseFunctions.instance
    .httpsCallable('verifyPurchase');

final res = await fn.call({
    'token': purchaseToken,
});
```

The auth token travels with the call, so the function knows who is asking without you passing a uid, and without trusting one the client sent.

Firebase is a backend you configure; Go is one you write. Most projects start with the first and move the parts it cannot cover to the second.

Before next week



Recap

The client is untrusted input; the server is the only place a rule actually holds

Serverless bills per request and cold-starts; a VPS bills per month and never does

Safe $\neq$ idempotent: GET and HEAD are safe, PUT and DELETE are only idempotent

Backoff without jitter is still a thundering herd

Generate JSON; never hand-write a fromJson again



This week

Run the Go handler locally and call it from your app with dio

Add explicit timeouts and one retry wrapper to every request in your project

Convert one hand-written model to @JsonSerializable and run build_runner

Open your Firestore rules and check every one compares request.auth.uid



Read more

go.dev/doc/tutorial/web-service-gin · pkg.go.dev/net/http

pub.dev/packages/dio · pub.dev/packages/json_serializable

firebase.google.com/docs/flutter/setup

developer.mozilla.org · HTTP request methods

aws.amazon.com/builders-library · timeouts, retries and backoff with jitter